UNIT-2

1. C++ Tokens

In C++, tokens can be defined as the smallest building block of C++ programs that the compiler understands. It is the smallest unit of a program into which the compiler divides the code for further processing.

Types of Tokens in C++

We have 5 types of tokens each of which serves a specific purpose in the syntax and semantics of C++. Below are the main types of tokens in C++:

1. Identifiers

In C++, entities like variables, functions, classes, or structs must be given unique names within the program so that they can be uniquely identified.

It is recommended to choose valid and relevant names of identifiers to write readable and maintainable programs. Keywords cannot be used as an identifier because they are reserved words to do specific tasks. In the below example, "first_name' is an identifier.

string first_name = "Raju";

We have to follow a set of rules to define the name of identifiers as follows:

1. An identifier can only begin with a letter or an underscore (_).
2. An identifier can consist of letters (A-Z or a-z), digits (0-9), and underscores (_). White spaces and Special characters can not be used as the name of an identifier.
3. Keywords cannot be used as an identifier because they are reserved words to do specific tasks. For example, string, int, class, struct, etc.
4. Identifier must be unique in its namespace.
5. As C++ is a case-sensitive language so identifiers such as 'first_name' and 'First_name' are different entities.

2. Keywords

Keywords in C++ are the tokens that are the reserved words in programming languages that have their specific meaning and functionalities within a program. Keywords cannot be used as an identifier to name any variables.

For example, a variable or function cannot be named as 'int' because it is reserved for declaring integer data type.

There are 95 keywords reserved in C++.

3. Constants

Constants are the tokens in C++ that are used to define variables at the time of initialization and the assigned value cannot be changed after that. We can define the constants in C++ in two ways that are using the const, constexpr keyword and #define preprocessor directive.

const type name = val
constexpr type name = val
#define name val

4. Strings

In C++, a string is not a built-in data type like 'int', 'char', or 'float'. It is a class available in the STL library which provides the functionality to work with a sequence of characters, that represents a string of text.

string variable_name;
string name= "This is string";

When we define any variable using the 'string' keyword we are actually defining an object that represents a sequence of characters. We can perform various methods on the string provided by the string class such as length (), push_backk(), and pop_back().

5. Punctuators

Punctuators are the token characters having specific meanings within the syntax of the programming language. These symbols are used in a variety of functions, including ending the statements, defining control statements, separating items, and more.

Below are the most common punctuators used in C++ programming:

(i) Semicolon (;): It is used to terminate the statement.

(ii) (Square brackets []: They are used to store array elements.

(iii) Curly Braces {}: They are used to define blocks of code.

(iv) Double-quote ("): It is used to enclose string literals.

(v) Single-quote ('): It is used to enclose character literals.

2. C++ Variables

In C++, **variable** is a name given to a memory location. It is the basic unit of storage in a program. The value stored in a variable can be accessed or changed during program execution.

2.1 Creating a Variable

Creating a variable and giving it a name is called **variable definition** (sometimes called **variable declaration**). The syntax of variable definition is:

type name;

where, **type** is the type of data that a variable can store, and **name** is the name assigned to the variable. Multiple variables of the same type can be defined as:

type name1, name2, name3.....;

These data types of a variable is selected from the list of data types supported by C++.

Example:

To store number without decimal point, we use integer data type.

int num;

Here, **int** is the keyword used to tell the compiler that the variable with name **num** will store integer values.

2.2 Initializing

A variable that is just defined may not contain some valid value. We have to initialize it to some valid initial value. It is done by using an assignment operator = as shown:

int num;

num = 3;

Definition and initialization can also be done in a single step as shown:

int num = 3;

The integer variable **num** is initialized with the value 3. Values are generally the literals of the same type.

2.3 Accessing and Updating

The main objective of a variable is to store the data so that it can be retrieved or update any time. Accessing can be done by simply using its assigned name and updating the value using **= assignment operator**.

Example:

```
#include <iostream>
```

```
using namespace std;
int main() {
    // Creating a single character variable
    int num = 3;
    // Accessing and printing above variable
    cout << num << endl;
    // Update the value
    num = 7;

    cout << num;
    return 0;
}
```

Output

```
3
7
```

2.4 Rules For Naming Variable

The names given to a variable are called identifers. There are some rules for creating these identifiers (names):

(i) A name can only contain **letters** (A-Z or a-z), **digits** (0-9), and **underscores (_)**.

(ii) It should start with a **letter or an underscore only.**

(iii) It is case sensitive.

(iv) The name of the variable should not contain any whitespace and special characters (i.e. #, $, %, *, etc).

(v) We cannot use C++ Keywords (e.g. float, double, class) as a variable name.

2.5 How are variables used?

Variables are the names given to the memory location which stores some value. These names can be used in any place where the value it stores can be used. **For example,** we assign values of the same type to variables. But instead of these values, we can also use variables that store these values.

```
#include <iostream>
using namespace std;
int main() {
    int num1 = 10, num2;
    // Assigning num1's value to num2
    num2 = num1;
    cout << num1 << " " << num2;
    return 0;
}
```

Output

```
10 10
```

Addition of two integers can be done in C++ using operator as shown:

```
#include <iostream>
using namespace std;
int main() {
    cout << 10 + 20;
    return 0;
}
```

Output

```
30
```

We can do the above operation using the variables that store these two values.

```
#include <iostream>
using namespace std;
int main() {
```

```
    int num1 = 10, num2 = 20;
    cout << num1 + num2;
    return 0;
}
```

Output

```
30
```

2.6 Constant Variables

In C++, a constant variable is one whose value cannot be changed after it is initialized. This is done using the const keyword.

```
#include <iostream>
using namespace std;
int main() {
    const int num = 10;
    cout << num;
    return 0;
}
```

2.7 Scope of Variables

Scope Variable is the region inside the program where the variable can be referred to by using its name. Basically, it is the part of the program where the variable exists. Proper understanding of this concept requires the understanding of other concepts such as functions, blocks, etc.

3. Data Types

1. Basic Data Types
   (i) int: Stores integers (whole numbers), typically 4 bytes.
   (ii) char: Stores a single character, usually 1 byte.

(iii) float: Stores single-precision floating-point numbers (decimal numbers), typically 4 bytes.

(iv) double: Stores double-precision floating-point numbers, usually 8 bytes.

(v) bool: Stores Boolean values (true or false).

2. Derived Data Types

(i) Pointer types: Store memory addresses (e.g., int*, char*).

(ii) Array types: Collection of elements of the same type (e.g., int arr[10];).

(iii) Function types: Types for functions returning a value.

3. User-Defined Data Types

(i) struct: Group of variables under one name.

(ii) class: Like struct but with access control (private, public).

(iii) union: A special data type where all members share the same memory location.

(iv) enum: User-defined list of named integral constants.

4. Modifiers

4.1 You can modify basic types using:

(i) signed / unsigned: Whether the integer can be negative or not.

(ii) short / long: Size variations (e.g., short int, long int).

Example:

```
int a = 10;

char c = 'A';

float f = 3.14f;

double d = 3.14159265359;

bool flag = true;
```

Summary Table:

Type	Description	Typical Size
int	Integer numbers	4 bytes
char	Single character	1 byte
float	Single precision floating-point	4 bytes
double	Double precision floating-point	8 bytes
bool	Boolean (true/false)	1 byte
void	No value	0 bytes

4. Operators

C++ Operators are special symbols that are used to perform operations on operands such as variables, constants, or expressions. A wide range of operators is available in C++ to perform a specific type of operations which includes arithmetic operations, comparison operations, logical operations, and more.

For example, `(A+B)`, in which 'A' and 'B' are operands, and '+' is an arithmetic operator which is used to add two operands.

There can be three types of operators:

1. Unary Operators

Unary operators are used with single operands only. They perform the operations on a single variable. For example, increment and decrement operators.

- Increment operator `(++)`: It is used to increment the value of an operand by 1.
- Decrement operator `(--)`: It is used to decrement the value of an operand by 1.

2. Binary Operators

They are used with the two operands and they perform the operations between the two variables. For example `(A<B)`, less than `(<)` operator compares A and B, returns true if A is less than B else returns false.

- Arithmetic Operators: These operators perform basic arithmetic operations on operands. They include `'+', '-', '*', '/',` and `'%'`
- Comparison Operators: These operators are used to compare two operands, and they include `'==', '!=', '<', '>', '<=',` and `'>='.`
- Logical Operators: These operators perform logical operations on boolean values. They include `'&&', '||',` and `'!'.`
- Assignment Operators: These operators are used to assign values to variables, and they include 'variables', `'-=', '*=', '/=',` and `'%='.`
- Bitwise Operators: These operators perform bitwise operations on integers. They include`'&', '|', '^', '~', '<<'and '>>'.`

3. Ternary Operator

The ternary operator is the only operator that takes three operands. It is also known as a conditional operator that is used for conditional expressions.

Expression1 ? Expression2 : Expression3;

If Expression1 became true then Expression2 will be executed
otherwise Expression3 will be executed.

Operator Precedence and Associativity in C++

In C++,operator precedence and associativity are important concepts that determine the order in which operators are evaluated in an expression. Operator precedence tells the priority of operators, while associativity determines the order of evaluation when multiple operators of the same precedence level are present.

4.1 C++ Operator Types

C++ operators are classified into **6 types** on the basis of type of operation they perform:

1. Arithmetic Operators

Arithmetic Operators are used to perform arithmetic or mathematical operations on the operands. For example, '**+**' is used for addition.

Operator	Description	Examples
+	Addition	a+b
-	Subtraction	a-b
*	Multiplication	a*b
/	Division	a/b
%	Modulus	a%b
++	Increment	a++
--	Decrement	a--

Example:

```
#include <iostream>
using namespace std;
int main() {
    int a = 8, b = 3;
    // Addition
    cout << "a + b = " << (a + b) << endl;
    // Subtraction
    cout << "a - b = " << (a - b) << endl;
    // Multiplication
    cout << "a * b = " << (a * b) << endl;
    // Division
    cout << "a / b = " << (a / b) << endl;
    // Modulo
    cout << "a % b = " << (a % b) << endl;

    // Increament
    cout << "++a = " << ++a << endl;
    // Decrement
    cout << "b-- = " << b--;
```

```
    return 0;
}
```

Output

```
a + b = 11
a - b = 5
a * b = 24
a / b = 2
a % b = 2
++a = 9
--b = 2
```

Important Points:

- The Modulo Operator (%) operator should only be used with integers. Other operators can also be used with floating point values.
- **++a** and **a++**, both are increment operators, however, both are slightly different. In **++a**, the value of the variable is incremented first and then it is used in the program. In **a++**, the value of the variable is assigned first and then it is incremented. Similarly happens for the decrement operator.

You may have noticed that some operator works on two operands while other work on one. On the basis of this operators are also classified as:

- Unary: Works on single operand.
- Binary: Works on two operands.
- Ternary: Works on three operands.

2. Relational Operators

Relational Operator are used for the comparison of the values of two operands. For example, '>' check right operand is greater.

Name	Symbol	Description
Is Equal To	**==**	Checks both operands are equal
Greater Than	**>**	Checks first operand is greater than the second operand
Greater Than or Equal To	**>=**	Checks first operand is greater than equal to the second operand
Less Than	**<**	Checks first operand is lesser than the second operand
Less Than or Equal To	**<=**	Checks first operand is lesser than equal to the second operand
Not Equal To	**!=**	Checks both operands are not equal

Example

```
#include <iostream>
using namespace std;
int main() {
    int a = 6, b = 4;
    // Equal operator
    cout << "a == b is " << (a == b) << endl;
    // Greater than operator
    cout << "a > b is " << (a > b) << endl;
    // Greater than Equal to operator
    cout << "a >= b is " << (a >= b) << endl;
    //  Lesser than operator
    cout << "a < b is " << (a < b) << endl;
    // Lesser than Equal to operator
    cout << "a <= b is " << (a <= b) << endl;
    // Not equal to operator
```

```
    cout << "a != b is " << (a != b);
    return 0;
}
```

Output

```
a == b is 0
a > b is 1
a >= b is 1
a < b is 0
a <= b is 0
a != b is 1
```

Note: 0 denotes false and 1 denotes true.

3. Logical Operators

Logical Operators are used to combine two or more conditions or constraints or to complement the evaluation of the original condition in consideration. The result returns a Boolean value, i.e., true or false.

Name	Symbol	Description
Logical AND	**&&**	Returns true only if all the operands are true or non-zero.
Logical OR	**||**	Returns true if either of the operands is true or non-zero.
Logical NOT	**!**	Returns true if the operand is false or zero.

Example:

```
#include <iostream>
using namespace std;

int main() {
    int a = 6, b = 4;
    // Logical AND operator
    cout << "a && b is " << (a && b) << endl;
```

```
    // Logical OR operator
    cout << "a || b is " << (a || b) << endl;
    // Logical NOT operator
    cout << "!b is " << (!b);
    return 0;
}
```

Output

```
a && b is 1
a || b is 1
!b is 0
```

4. Bitwise Operators

Bitwise Operators are works on bit-level. So, compiler first converted to bit-level and then the calculation is performed on the operands.

Name	Symbol	Description
Binary AND	**&**	Copies a bit to the evaluated result if it exists in both operands
Binary OR	**|**	Copies a bit to the evaluated result if it exists in any of the operand
Binary XOR	**^**	Copies the bit to the evaluated result if it is present in either of the operands but not both
Left Shift	**<<**	Shifts the value to left by the number of bits specified by the right operand.
Right Shift	**>>**	Shifts the value to right by the number of bits specified by the right operand.
One's Complement	**~**	Changes binary digits 1 to 0 and 0 to 1

Example:

```
#include <iostream>
using namespace std;
int main() {
    int a = 6, b = 4;

    // Binary AND operator
    cout << "a & b is " << (a & b) << endl;

    // Binary OR operator
    cout << "a | b is " << (a | b) << endl;

    // Binary XOR operator
    cout << "a ^ b is " << (a ^ b) << endl;

    // Left Shift operator
    cout << "a << 1 is " << (a << 1) << endl;

    // Right Shift operator
    cout << "a >> 1 is " << (a >> 1) << endl;

    // One’s Complement operator

    cout << "~(a) is " << ~(a);

    return 0;

}
```

Output

```
a & b is 4

a | b is 6

a ^ b is 2

a<<1 is 12

a>>1 is 3

~(a) is -7
```

5. Assignment Operators

Assignment operators are used to assign value to a variable. We assign the value of right operand into left operand according to which assignment operator we use.

Name	Symbol	Description
Assignment	**=**	Assigns the value on the right to the variable on the left.
Add and Assignment	**+=**	First add right operand value into left operand then assign that value into left operand.
Subtract and Assignment	**-=**	First subtract right operand value into left operand then assign that value into left operand.
Multiply and Assignment	***=**	First multiply right operand value into left operand then assign that value into left operand.
Divide and Assignment	**/=**	First divide right operand value into left operand then assign that value into left operand.

Example:

```
#include <iosteam>
using namespace std;
int main() {
    int a = 6, b = 4;
    // Assignment Operator.
    cout << "a = " << a << endl;
    //  Add and Assignment Operator.
    cout << "a += b is " << (a += b) << endl;
    // Subtract and Assignment Operator.
    cout << "a -= b is " << (a -= b) << endl;
    //  Multiply and Assignment Operator.
    cout << "a *= b is " << (a *= b) << endl;
    //  Divide and Assignment Operator.
    cout << "a /= b is " << (a /= b);

    return 0;
```

```
}
```

Output

```
a = 6

a += b is 10

a -= b is 6

a *= b is 24

a /= b is 6
```

6. Ternary or Conditional Operators

Conditional Operator returns the value, based on the condition. This operator takes **three operands**, therefore it is known as a **Ternary Operator**.

Syntax:

Expression1 ? Expression2 : Expression3

In the above statement:

- The ternary operator **?** determines the answer on the basis of the evaluation of **Expression1**.
- If **Expression1** is true, then **Expression2** gets evaluated.
- If **Expression1** is false, then **Expression3** gets evaluated.

Example:

```
#include <iostream>

using namespace std;

int main() {

    int a = 3, b = 4;

    // Conditional Operator

  int result = (a < b) ? b : a;

    cout << "The greatest number "

          "is " << result;

    return 0;

}
```

Output

```
The greatest number is 4
```

5. Usage of Math Library

The <cmath> (or <math.h> in C-style) library in C++ provides a wide range of mathematical functions. It is part of the C++ Standard Library and includes functions for:

(i) Basic mathematical operations (abs, pow, sqrt)

(ii) Trigonometry (sin, cos, tan)

(iii) Exponential and logarithmic functions (exp, log, log10)

(iv) Rounding and remainder functions (ceil, floor, fmod)

5.1 How to Use It

```
#include <iostream>
#include <cmath>  // math library
int main() {
    double x = 9.0;
    cout << "Square root of x: " << sqrt(x) << endl;
    cout << "Power: " << pow(x, 2) << endl;
    cout << "Sine of x: " << sin(x) << endl;
    cout << "Cosine of x: " << cos(x) << endl;
    cout << "Exponential of x: " << exp(x) << endl;
    cout << "Natural log of x: " << log(x) << endl;
    cout << "Ceiling of x: " << ceil(x) << endl;
    cout << "Floor of x: " << floor(x) << endl;
    cout << "Absolute value: " << fabs(-x) << endl;
    return 0;
}
```

5.2 Common Functions in <cmath>

Function	Description

sqrt(x)	Square root
pow(x, y)	x raised to the power y
abs(x)	Absolute value (int)
fabs(x)	Absolute value (floating point)
sin(x)	Sine (x in radians)
cos(x)	Cosine
tan(x)	Tangent
asin(x)	Arc sine
acos(x)	Arc cosine
atan(x)	Arc tangent
exp(x)	e^x
log(x)	Natural logarithm (base e)
log10(x)	Logarithm base 10
ceil(x)	Rounds up
floor(x)	Rounds down
fmod(x, y)	Floating-point remainder (x mod y)
round(x)	Rounds to nearest integer
hypot(x, y)	sqrt($x^2 + y^2$)

5.3 Headers: <cmath> vs <math.h>

(i) <cmath> is the C++ version and puts functions in the std namespace.

(ii) <math.h> is the C-style header (still usable in C++) and places functions in the global namespace.

5.4 C++ style usage:

```
#include <cmath>
cout << sqrt(25);
```

C style usage:

```
#include <math.h>
```

```
printf("%f", sqrt(25));
```

5. Type Conversion

Type conversion in C refers to the **implicit conversion** of a value from one data type to another.

In type conversion, a data type is automatically converted into another data type by a compiler at the compiler time.

In type conversion, the **destination data type cannot be smaller than the source data type**, that's why it is also called **widening conversion**.

One more important thing is that it can only be applied to compatible data types.

For example:

```
int x=30;
float y;
  y=x;
```

6. Typecasting

Typecasting refers to the **explicit conversion** of a variable from one data type to another by the programmer.

In typing casting, a data type is converted into another data type by the programmer using the **casting operator** during the program design.

In typecasting, the **destination data type may be smaller than the source data type** when converting the data type to another data type, that's why it is also called **narrowing conversion**.

For example:

```
double num1 = 3.14;
int num2 = (int)num1; // Typecasting: double to int
```

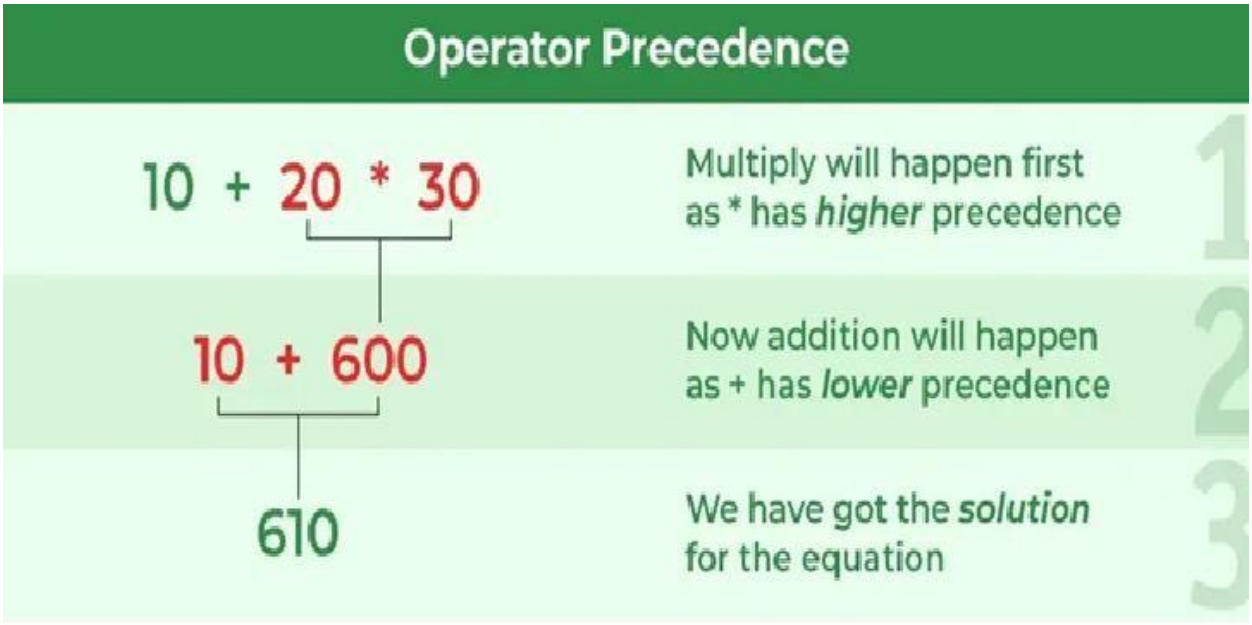
7. Operator Precedence in C++

In C++, operator precedence specifies the order in which operations are performed within an expression. When an expression contains multiple operators, those with higher precedence are evaluated before those with lower precedence.

For expression:

10 + 20 * 30

The expression contains two operators, + (addition) and * (multiplication). According to operator precedence, multiplication (*) has higher precedence than addition (+), so multiplication is checked first. After evaluating multiplication, the addition operator is then evaluated to give the final result.



Example of C++ Operator Precedence

```
#include <iostream>

using namespace std;

int main(){

    // Multiplication has higher precedence

    int result = 10 + 20 * 30;

    cout << "Result: " << result << endl;

    return 0;

}
```

Output

```
Result: 610
```

8. Operator Associativity in C++

Operator associativity determines the order in which operands are grouped when multiple operators have the same precedence. There are two types of associativity:

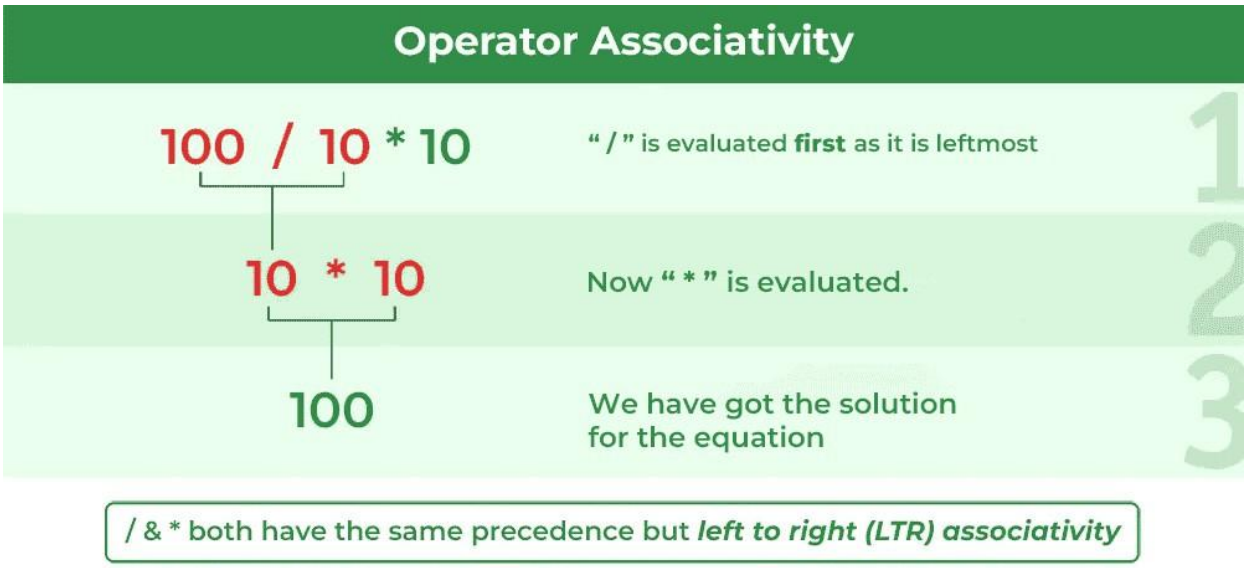
(i) **Left-to-right associativity** means that when multiple operators of the same precedence appear in an expression, they are evaluated from left to right. For example, in the expression a + b - c, addition and subtraction have the same precedence and are left-associative, so the expression is evaluated as (a + b) - c.

(ii) **Right-to-left associativity** means the operators are evaluated from right to left. For example, the assignment operator = is right-associative. So, in the expression a = b = 4, the value 4 is first assigned to b, and then the result of that assignment (b, which is now 4) is assigned to a.

For expression:

100 / 10 % 10

The division (/) and modulus (%) operators have the same precedence, so the order in which they are evaluated depends on their left-to-right associativity. This means the division is performed first, followed by the modulus operation. After the calculations, the result of the modulus operation is determined.




```
#include <iostream>
```

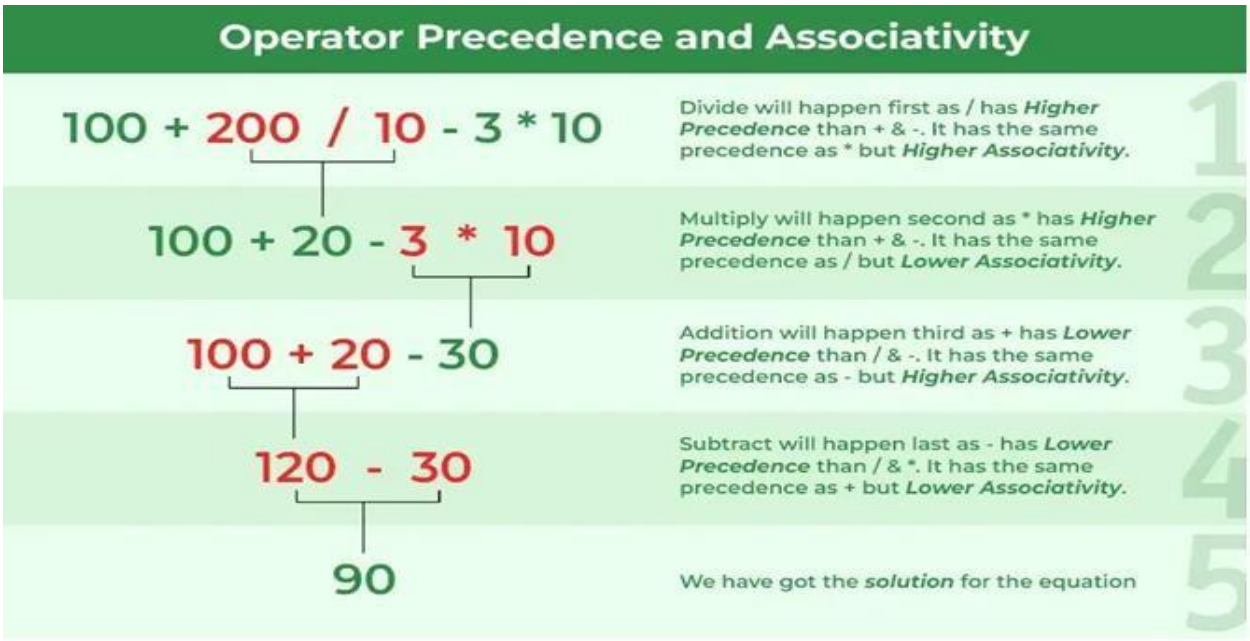
```
using namespace std;
int main() {
    int result = 100 / 10 *10;
    cout<<"Result: "<< result<<"\n";
    return 0;
}
```

Output

```
Result: 100
```

Example of C++ Operator Precedence and Associativity

In general, operator precedence and associativity are applied together in expressions. Consider the expression **exp = 100 + 200 / 10 - 3 * 10**, where the division (/) and multiplication (*) operators have the same precedence but are evaluated before addition (+) and subtraction (-). Due to left-to-right associativity, the division is evaluated first, followed by multiplication. After evaluating the division and multiplication, the addition and subtraction are evaluated from left to right, giving the final result.




```
#include <iostream>
using namespace std;
int main() {
```

```
    int result = 100 + 200 / 10 - 3 * 10;
    // Verifying the result of the same expression
    cout << "Result: " << result << endl;
    return 0;
}
```

Output

```
Result: 90
```

9. Operator Precedence Table

The operator precedence table in C++ is a table that lists the operators in order of their precedence level. Operators with higher precedence are evaluated before operators with lower precedence. This table also includes the associativity of the operators, which determines the order in which operators of the same precedence are processed.

The operators are listed from top to bottom, in descending precedence

Operator	Name	Associativity
() [] -> .	Function call, Subscript, Member access	Left
++ --	Increment/Decrement	Right
! ~ - +	Logical/Bitwise NOT, Unary plus/minus	Right
*** / %**	Multiplication, Division, Modulus	Left
+ -	Addition, Subtraction	Left
<< >>	Bitwise shift	Left
< <= > >=	Relational operators	Left
== !=	Equality operators	Left
&	Bitwise AND	Left
^	Bitwise XOR	Left
|	Bitwise OR	Left
&&	Logical AND	Left
||	Logical OR	Left
?:	Ternary conditional	Right
= += -= *= /= %= &= ^= |= <<= >>=	Assignment and compound assignment	Right
,	Comma	Left

10. Control Flow Statements.

Control flow refers to the order in which statements within a program execute. While programs typically follow a sequential flow from top to bottom, there are scenarios where we need more flexibility. This article provides a clear understanding about everything you need to know about Control Flow Statements.

10.1 What are Control Flow Statements in Programming?

Control flow statements are fundamental components of programming languages that allow developers to control the order in which instructions are executed in a program. They enable execution of a block of code multiple times, execute a block of code based on conditions, terminate or skip the execution of certain lines of code, etc.

10.2 Types of Control Flow statements in Programming:

Control Flow Statements Type	Control Flow Statement	Description
Conditional Statements	if-else	Executes a block of code if a specified condition is true, and another block if the condition is false.
	switch-case	Evaluates a variable or expression and executes code based on matching cases.
Jump Statements	break	Terminates the loop or switch statement and transfers control to the statement immediately following the loop or switch.
	continue	Skips the current iteration of a loop and continues with the next iteration.

11.Conditional Statements in Programming:

Conditional statements in programming are used to execute certain blocks of code based on specified conditions. They are fundamental to decision-making in programs. Here are some common types of conditional statements:

1. If Statement in Programming:

The if statement is used to execute a block of code if a specified condition is true.

```
#include <stdio.h>

int main(){

    int  a  =  5;

    if (a == 5) {

        cout <<"a is equal to 5"<< endl”;

    }

    return 0;
```

```
}
Output

a is equal to 5
```

2. if-else Statement in Programming:

The if-else statement is used to execute one block of code if a specified condition is true, and another block of code if the condition is false.

```
#include <stdio.h>

int main(){

    int a = 10;

    if (a == 5) {

        cout<<"a is equal to 5"<< endl;

    }

    else {

        cout<<"a is not equal to 5"<<;

    }

    return 0;

}


Output

a is not equal to 5
```

3. if-else-if Statement in Programming:

The if-else-if statement is used to execute one block of code if a specified condition is true, another block of code if another condition is true, and a default block of code if none of the conditions are true.

```
#include <stdio.h>

int main(){

    int a = 15;

    if (a == 5) {

        printf("a is equal to 5");

    }

    else if (a == 10) {

        printf("a is equal to 10");
```

```
    }
    else {
        printf("a is not equal to 5 or 10");
    }
    return 0;
}
```

```
Output
a is not equal to 5 or 10
```

4. Ternary Operator or Conditional Operator in Programming:

In some programming languages, a ternary operator is used to assign a value to a variable based on a condition.

```
#include <stdio.h>
int main(){
    int a = 10;
    printf("%s", (a == 5 ? "a is equal to 5"
                         : "a is not equal to 5"));
    return 0;
}
```

```
Output
a is not equal to 5
```

5. Switch Statement in Programming:

In languages like C, C++, and Java, a switch statement is used to execute one block of code from multiple options based on the value of an expression.

```
#include <stdio.h>
int main(){

    int a = 15;
    switch (a)
    { case 5:
```

```
        printf("a is equal to 5");
        break;
    case 10:
        printf("a is equal to 10");
        break;
    default:
        printf("a is not equal to 5 or 10");
    }
    return 0;
}
```

Output

```
a is not equal to 5 or 10
```